BAHAR WEEKLY MISSION 02 / STUDENT EDITION

مأموریت هفتگی شماره ۰۲

از عدد تا رجیستر؛ فهم و کنترل بیت‌ها

نمایش دودویی و مبنای شانزده، ماسک بیتی و تغییر ایمن یک رجیستر فرضی با زبان C

چه زمانی موفق شده‌ای؟

وقتی **قبل از اجرای کد** مقدار نهایی را پیش‌بینی کنی، سپس نتیجه را در اشکال‌زدا یا آزمون ببینی و اختلاف‌ها را توضیح بدهی.

این هفته چه می‌سازیم؟

یک «شبیه‌ساز رجیستر هشت‌بیتی» که بتواند فقط بیت‌های خواسته‌شده را روشن، خاموش یا معکوس کند و رفتار آن با آزمون اثبات شود.

مشخصه	مقدار
شناسه	NSP-TR-BH-W02-M02
نسخه	۱.۲ – افزودن دستور عملی Git، آزمون‌های قابل تکرار و رفع ابهام‌های مأموریت ۰۱
بار تقریبی	۸ تا ۱۰ ساعت برای بخش اصلی، با زمان‌بندی شناور؛ تکمیل موارد هفته اول در بازه‌های کوتاه جداگانه
پیش‌نیاز	آشنایی اولیه با محیط C و Debugger؛ ابهام‌های Git و ایمنی در «پل کوتاه هفته اول» پیگیری می‌شوند.
سخت‌افزار الزامی	خیر؛ کامپیوتر، کامپایلر C و Debugger کافی است. استفاده از برد واقعی اختیاری و با هدف مشخص است.

این سند نسبت به نسخه قبلی روشن‌تر شده است: واژه‌های فنی در همان محل تعریف می‌شوند؛ برای هر کار یک مثال و یک خروجی قابل بررسی داریم؛ معلوم است چه کسی چالش را ارائه می‌دهد؛ و «ساخته‌ام» با «توانسته‌ام اثبات کنم» اشتباه گرفته نمی‌شود.

NSP ELECTRONICS | LEARN - BUILD - VERIFY

۱. پل کوتاه از مأموریت اول

این بخش «جبران تنبیهی» نیست؛ فقط سه ابهامی را که در بازخورد قبلی مطرح شد روشن می‌کند. اجرای اصلی مأموریت دوم نباید به خاطر واژه‌های مبهم عقب بیفتد.

الف) تفاوت Git و GitHub در ۵ دقیقه

Git تاریخچه نسخه‌های پروژه را روی کامپیوتر ثبت می‌کند؛ **GitHub** می‌تواند نسخه‌ای از مخزن را آنلاین نگه دارد؛ **Commit** یک نقطه مشخص در تاریخچه است. مخزن لازم نیست منتظر محصول کامل بماند: همان فایل C، تنظیمات اصلی IAR و فایل توضیحی می‌توانند اولین Commit باشند.

```
Bahar-Embedded-Track/
  README.md
  week01/
    src/main.c
    project/  (IAR project files)
    evidence/  (selected screenshots)
  week02/
    src/
    tests/
    evidence/
```

اگر مخزن آنلاین از قبل ساخته شده، از نسخه آنلاین Clone کن و فایل‌های تمرین را در همان پوشه قرار بده. فایل‌های خروجی Build، اطلاعات شخصی، کلیدها و فایل‌های محرمانه را منتشر نکن.

ب) «پروژه از مسیر تمیز دوباره ساخته شود» یعنی چه؟

یعنی با نسخه‌ای از فایل‌های **منبع** که خروجی‌های قبلیِ کامپایل در آن دخالت ندارند، بتوانی دوباره Build موفق بگیری. برای اثبات: از فایل‌های خودت نسخه پشتیبان بگیر، پوشه تازه‌ای از مخزن ایجاد کن، در IAR پروژه را باز کن و **Rebuild All** بزن. نتیجه Build را ذخیره کن. صرفاً اجرای پروژه‌ای که قبلاً ساخته شده کافی نیست.

ج) کار ایمن با برد – فعلاً فقط آشنایی، نه آزمون اجباری

قبل از اتصال هر برد واقعی باید **مدل و ولتاژ مجاز** از روی مستندات تعیین شود، اتصال کوتاه آشکار بین تغذیه و زمین بررسی شود و منبع تغذیه با **محدودیت جریان متناسب با همان برد** تنظیم شود. اگر مقدار مجاز یا اتصال‌ها مشخص نیست، **برد را روشن نکن**؛ با راهنما بررسی کن. در این هفته نیازی به روشن کردن برد نیست.

> **چه چیزی هنوز از هفته ۰۱ باز است؟** یک Commit قابل مشاهده، یک Rebuild از محل تازه، و توضیح شفاهی اصول تغذیه ایمن. این‌ها به‌صورت موارد انتقالی ثبت می‌شوند، نه اینکه تمام یادگیری هفته اول مردود اعلام شود.

راهنمای عملی ۱۵ دقیقه‌ای: اولین ثبت واقعی در Git

منظور از «مخزن» یک پوشه معمولی حاوی فایل‌های پروژه است که Git تاریخچه تغییراتش را نگهداری می‌کند. **برای اولین ثبت لازم نیست محصول یا بردی آماده شده باشد.** همان کد ++x و پروژه IAR کافی‌اند.

مرحله ۱ — فایل‌ها را مرتب کن

در مخزن Bahar-Embedded-Track پوشه week01/src را بساز و فایل main.c را آنجا قرار بده. پروژه IAR و تنظیمات ضروری را در week01/iar بگذار و یک توضیح کوتاه در README.md بنویس.

مرحله ۲ — تغییرات را ثبت کن

اگر با Git Bash کار می‌کنی، از پوشه اصلی مخزن، دستورهای زیر را اجرا کن. اگر با GitHub Desktop راحت‌تری، همان فایل‌ها را انتخاب کن، یک پیام Commit بده و سپس Push کن.

```
git status
git add README.md week01/
git commit -m "W01: first C-SPY debug lab"
git log --oneline -3
git push
```

اگر Git هنوز روی سیستم آماده نیست: ابتدا مخزن آنلاینی را که ساخته‌ای روی کامپیوتر Clone کن. اگر Commit یا Push خطا داد، متن دقیق خطا را ذخیره کن؛ لازم نیست حدسی چند تنظیم مختلف را تغییر بدهی. **/Output/Debug** و **Release** و فایل‌های ساخته‌شده توسط کامپایلر را به‌صورت پیش‌فرض وارد مخزن نکن.

مرحله ۳ — «بازسازی از مسیر تمیز» را نشان بده

در یک پوشه تازه، از مخزن Clone بگیر؛ پروژه IAR را از همان محل تازه باز کن؛ Project → Rebuild All را اجرا کن و از نتیجه Build (خطا/هشدار) تصویر بگیر. پوشه اصلی را حذف نکن. این آزمایش نشان می‌دهد برای ساخت، به فایل‌های مخفی یا خروجی‌های قدیمیِ فقط روی کامپیوتر قبلی وابسته نیستی.

تحویل کوچک و مشخص: یک تصویر از git log یا صفحه Commits، یک تصویر از نتیجه Rebuild در پوشه تازه، و دو جمله درباره نتیجه. **ثبت این شواهد، مهارت مستقل را اثبات می‌کند؛ صرف وجود آدرس GitHub به‌تنهایی کافی نیست.**

سه اصطلاحی که در دروازه هفته اول پرسیده شده بود: کامپایلر (Compiler) کد C را به فایل شیء تبدیل می‌کند؛ **لینکر (Linker)** بخش‌های ساخته‌شده را به هم متصل می‌کند و خروجی اجرایی می‌سازد؛ **اشکال‌زدا (Debugger)** اجرای همان خروجی را برای توقف، اجرای مرحله‌ای و مشاهده متغیرها بررسی می‌کند. هرکدام نقشی متفاوت دارند.

۲. زبان رجیستر را ساده ببین

رجیستر را فعلاً یک خانه فرضی با **۸ کلید مستقل** تصور کن؛ هر کلید یک «بیت» است و فقط ۰ یا ۱ می‌گیرد. در نمایش دودویی، بیت سمت راست **شماره ۰** و بیت سمت چپ **شماره ۷** است.

شماره بیت	۷	۶	۵	۴	۳	۲	۱	۰
مقدار در 0xA5	۱	۰	۱	۰	۰	۱	۰	۱

مثال حل‌شده، قبل از هر کدنویسی

مقدار اولیه 0xA5 یعنی 1010 0101. قرار است فقط بیت ۱ را یک کنیم. ماسک بیت ۱ می‌شود 0x02 = 0000 0010.
نتیجه پیش‌بینی‌شده: 0xA7 = 1010 0111. هیچ بیت دیگری نباید تغییر کند.

```
uint8_t reg = 0xA5;
reg = (uint8_t)(reg | 0x02u);   // Expected: 0xA7
```

نام	کاربرد	یادآوری
دودویی (Binary)	نمایش با صفر و یک	هر رقم یک بیت است.
مبنای شانزده (Hex)	نمایش کوتاه بیت‌ها با رقم‌های ۰–۹ و A–F	هر رقم Hex برابر ۴ بیت است.
ماسک (Mask)	مشخص می‌کند کدام بیت هدف عملیات است.	برای تغییر یک بیت، کل رجیستر را از نو ننویس.
عملگرهای بیتی	و، یا، انحصاری و نقیض: ~ ^ | &	با عملگرهای منطقی && || فرق دارند.
انتقال بیت (Shift)	<< >> برای جابه‌جایی بیت‌ها	پهنا، نوع داده و تقدم عملگرها مهم‌اند.

هشدار فنی: این هفته «رجیستر فرضی در RAM» را دستکاری می‌کنیم. روی رجیستر واقعی سخت‌افزار، خواندن-تغییر-نوشتن همیشه ایمن نیست؛ برخی بیت‌ها از نوع پاک‌شدن با نوشتن ۱، فقط‌نوشتنی یا دارای اثر جانبی هستند. در آینده رفتار هر رجیستر را از Reference Manual همان MCU بررسی می‌کنیم.

دام پر تکرار: یک uint8_t مقدار ۰ تا ۲۵۵ را نگه می‌دارد؛ ولی در عبارت‌های C ممکن است قبل از انتساب، **ارتقای نوع عدد صحیح (integer promotion)** رخ دهد. خروجی عبارت را با مقداری که دوباره در متغیر ۸ بیتی ذخیره می‌شود یکسان فرض نکن. برای عملگرهای بیتی ترجیحاً از انواع بدون علامت استفاده کن.

۳. ارزیابی پیش از شروع | حدود ۲۰ تا ۳۰ دقیقه

قبل از مطالعه کامل، با دانش فعلی پاسخ بده. سؤال تشخیصی امتحان نیست: «نمی‌دانم» کاملاً پذیرفتنی است. **پاسخ اولیه بدون AI و جست‌وجو** باشد؛ پس از ثبت می‌توانی منابع را بررسی کنی.

1) عدد ۱۷۳ دهدهی را به دودویی و Hex تبدیل کن؛ روش را بنویس.

2) در مقدار 0xA0 چند بیتِ یک وجود دارد و کدام‌اند؟

3) اگر reg = 0x52 باشد، نتیجه reg | 0x08 چیست؟

4) برای صفر کردن فقط بیت ۳ در یک رجیستر فرضی چه عبارتی می‌نویسی؟

5) تفاوت عملگرهای & و && را با یک مثال بگو.

6) تفاوت عملگرهای | و || را بگو.

7) عبارت 1u << 5 چه ماسکی می‌سازد؟

8) تفاوت signed و unsigned چیست؟ درباره سرریز چه می‌دانی؟

9) چرا در Embedded از uint8_t و uint16_t استفاده می‌کنیم؟

10) چرا ممکن است نوشتن یک عدد ثابت روی رجیستری با بیت‌های رزروشده خطرناک باشد؟

> **قالب تحویل:** شماره سؤال، پاسخ اولیه، میزان اطمینان ۰ تا ۵، زمان صرف‌شده، و مواردی که حدس زده‌ای. اگر چند مورد اصلی را بلد نبودی، با یک پیش‌تمرین ۳۰–۹۰ دقیقه‌ای وارد کار می‌شویم.

۴. نقشه اجرای مأموریت

موضوع‌ها ترتیب پیشنهادی دارند، اما ساعت اجرا شناور است. هر جلسه را می‌توان به دو بازه کوتاه تقسیم کرد. هدف «پر کردن ساعت» نیست؛ پایان هر مرحله یک نتیجه قابل مشاهده است.

مرحله	زمان	کاری که انجام می‌دهی و معیار پایان
۱: نمایش عدد	۷۵ دقیقه	۱۰ عدد را بین دهدهی، دودویی و Hex تبدیل کن؛ هر کدام را برعکس هم کنترل کن.
۲: نوع داده	۹۰ دقیقه	بازه‌های عددی، انواع stdint.h، سرریز و تفاوت signed/unsigned را با مثال بررسی کن.
۳: عملگر بیتی	۱۲۰ دقیقه	جدول درستیِ ~ ^ | & را بساز؛ ۶ ماسک واقعی محاسبه کن.
۴: جابه‌جایی بیت	۹۰ دقیقه	ماسک بیت‌های ۰ تا ۷ را با Shift بساز؛ ابتدا نتیجه را پیش‌بینی کن.
۵: شبیه‌ساز رجیستر	۱۵۰ دقیقه	چهار تابع Set/Clear/Toggle/Test بنویس؛ تست‌ها اجرا و با مقدار پیش‌بینی‌شده مقایسه شوند.
۶: چالش و توضیح	۷۵ دقیقه	مسئله جدید را نخست مستقل حل کن؛ سپس با راهنما نتیجه را بررسی کن.

اگر وسط کار گیر کردی چه کنیم؟

بن‌بست قابل توضیح

پس از دو فرضیه/تست ناموفق، وضعیت را ثبت کن، استراحت کوتاه کن و سپس یک Hint هدفمند بخواه.

پیشرفت کوچک

وقتی یک تبدیل، ماسک یا تست درست شد، همان مرحله را ثبت کن و سراغ بعدی برو.

دست‌کم سه نقطه پیشرفت قابل مشاهده در هفته ثبت شود: «تبدیل درست»، «تابع درست»، «تست موفق». اگر واقعاً از برنامه عقب افتادی، بار را جابه‌جا می‌کنیم، نه اینکه کارهای ناقص را بدون پیش‌نیاز روی هم جمع کنیم.

۵. سه بسته کاری اصلی، با تعریف «تمام‌شدن»

وظیفه ۱ — بفهم | تبدیل و استدلال بیتی

درخواست: ۲۰ عدد مختلف شامل دهدهی، دودویی و Hex انتخاب کن و هرکدام را به دو شکل دیگر تبدیل کن. دست‌کم ۵ مورد باید جایگاه بیت یا محاسبه ماسک داشته باشند.

مثال: برای عدد 0x2C بنویس کدام بیت‌ها یک‌اند و چرا حاصل در دهدهی ۴۴ است.

تحویل: فایل week02/notes/conversions.md با مراحل حل و کنترل. پاسخ نهایی بدون روش محاسبه کافی نیست.

وظیفه ۲ — بساز | چهار تابع C

درخواست: روی متغیر بدون علامت چهار تابع set_bits، clear_bits، toggle_bits و test_bits بساز. هر تابع حداقل ۵ آزمون داشته باشد؛ شامل ماسک صفر، بیت ۰ و بیت ۷.

قرارداد آزمون: مشخص کن ورودی چیست، خروجی مورد انتظار چیست، خروجی واقعی چیست و چرا سایر بیت‌ها تغییر نمی‌کنند. برای test_bits صریح تعریف کن نتیجه «همه بیت‌های ماسک یک‌اند» است یا «حداقل یکی یک است».

تحویل: bit_ops.c/.h و جدول حداقل ۲۰ آزمون.

وظیفه ۳ — اثبات کن | شبیه‌ساز رجیستر

درخواست: یک رجیستر فرضی uint8_t تعریف کن. حداقل ۶ تغییر پشت‌سرهم روی آن انجام بده. قبل از اجرای هر خط مقدار دودویی/Hex بعدی را پیش‌بینی کن؛ سپس در Debugger یا خروجی آزمون مشاهده کن.

تحویل: جدول «قبل، دستور، انتظار، مشاهده، نتیجه» و دو تصویر مستند یا فایل خروجی تست. اگر انتظار و مشاهده اختلاف داشتند، فرضیه و آزمون بعدی را بنویس.

قبل (Hex)	دستور/ماسک	انتظار (Hex)	مشاهده	توضیح
0xA5	Set bit 1	0xA7	پس از اجرا تکمیل شود	فقط بیت ۱ از ۰ به ۱ تغییر می‌کند.
...	...	...	...	...

کارگاه آزمون: چطور مطمئن شوم بیت دیگری تغییر نکرده؟

حالا که یک تابع ساخته‌ای، باید بتوانی ثابت کنی تنها بیت‌های داخل ماسک تغییر کرده‌اند. این مثال صرفاً قالب آزمون را نشان می‌دهد؛ جواب چهار تابع اصلی را به‌جای تو نمی‌نویسد.

مثال بررسی‌شده: مقدار اولیه 0xA5 است و فقط بیت ۱ را روشن می‌کنی. مقدار نهایی باید 0xA7 شود. اگر مقدار خروجی چیز دیگری بود، از روی جدول بیت‌ها بررسی کن کدام بیت تغییر ناخواسته کرده است.

```
#include <stdint.h>
uint8_t before = 0xA5u;
uint8_t mask   = (uint8_t)(1u << 1);
uint8_t after  = (uint8_t)(before | mask);
/* Predict after before running: 0xA7 */
```

الگوی جدول آزمون؛ همین قالب را برای توابع خودت ادامه بده

ورودی		ماسک	عملیات	انتظار	واقعی	بیت‌های بیرون ماسک
0xA5	0x02	Set	0xA7	پس از اجرا	باید ثابت بمانند	
0x00	0x00	Toggle	قبل از اجرا بنویس	پس از اجرا	توضیح بده	
0xFF	0x80	Clear	قبل از اجرا بنویس	پس از اجرا	توضیح بده	

درباره تابع Test دقیق باش

وقتی ماسک چند بیت دارد، تابع test_bits باید یک معنی مشخص داشته باشد: **آیا همه بیت‌های ماسک روشن‌اند؟** یا **آیا دست‌کم یکی روشن است؟** این دو رفتار متفاوت‌اند. یکی را در توضیح API انتخاب کن؛ برای mask=0 هم رفتار تابع را تعریف و آزمون کن.

در محیط IAR: ابتدا از تنظیمات Debug با Optimization مناسب مشاهده متغیر استفاده کن. اگر متغیری در Watch دیده نشد، فوراً آن را «خرابی C» تلقی نکن؛ تنظیمات بهینه‌سازی و دامنه عمر متغیر را بررسی کن. بحث دقیق volatile در هفته سوم است.

نقطه پایان این صفحه: برای چهار تابع حداقل ۵ آزمون معنادار بنویس؛ مجموعاً حداقل ۲۰ آزمون. هر آزمون باید ورودی، انتظار، مشاهده و نتیجه «قبول/رد» داشته باشد. صرفاً افزودن بیست خط مشابه بدون پوشش حالت‌های مرزی هدف نیست.

۶. چالش ناشناخته دقیقاً چیست و چه کسی آن را می‌دهد؟

چالش ناشناخته مسئله تازه‌ای است که جزئیاتش از قبل در مأموریت نوشته نمی‌شود؛ چون هدف سنجش انتقال دانش است، نه تکرار جواب آماده.

نقش	کار مشخص
راهنما / Mentor	شخصی که با تو درباره تکلیف گفت‌وگو و نتیجه را ارزیابی می‌کند؛ برای این برنامه مسئول هماهنگی پروژه است. او **پس از اتمام وظایف اصلی** یک رجیستر فرضی با وضعیت اولیه و شرایط تازه می‌دهد.
بهار	قبل از AI، ماسک و مقدار نهایی را روی کاغذ پیش‌بینی می‌کند، کد می‌نویسد، اجرا می‌کند و شواهد ارائه می‌دهد.
اگر گیر کردی	اول دو فرضیه یا تلاش را ثبت کن. سپس فقط سؤال راهنما بگیر، نه کد کامل. پاسخ اولیه مستقل بماند.
پس از ثبت	استفاده از AI و منابع برای نقد و مقایسه مجاز است؛ برای تأیید نهایی باید تست یا Debugger داشته باشی.

نمونه تمرینیِ آشکار (چالش اصلی نیست):

یک رجیستر با مقدار 0x35 داریم. بیت ۷ را ۱ کن و بیت ۲ را ۰ کن، سایر بیت‌ها را حفظ کن. مقدار نهایی را پیش‌بینی کن و بعد امتحان کن. این نمونه صرفاً برای فهم قالب درخواست است.

نکته مهم: هیچ قرار نیست خودت حدس بزنی راهنما چه سؤال مخفی‌ای در ذهن دارد. تا وقتی صورت مسئله تازه به تو تحویل نشده، این بخش «انجام‌نشده به علت ارائه‌نشدن مسئله» است؛ نه مردودی.

ثبت کوتاه مسیر حل

۱) مشاهده یا درخواست دقیق چه بود؟ ۲) فرضیه/برنامه من چه بود؟ ۳) چه آزمایشی کردم؟ ۴) چه نتیجه‌ای گرفتم؟ ۵) کدام بیت‌ها باید ثابت می‌ماندند و از کجا می‌دانم؟

۷. معیار عبور و تحویل نهایی

پایان هفته به معنی عبور خودکار نیست. امتیاز کلی ۱۰۰ است و هر مورد باید با شواهد فنی پشتیبانی شود.

معیار	امتیاز	چطور اثبات می‌شود؟
نمایش اعداد و جایگاه بیت	۱۵	تبدیل و توضیح مستقل در چند مثال ۸/۱۶ بیتی
ماسک و تغییر ایمن	۲۵	توابع و آزمون‌ها؛ حفظ بیت‌های خارج از ماسک
انواع عددی و سرریز	۲۰	توضیح signed/unsigned و محدودیت Shift
پیش‌بینی و Debug	۲۰	جدول انتظار در برابر مشاهده و تحلیل اختلاف
چالش ناشناخته	۲۰	حل اولیه مستقل و ثبت استدلال و تست

۸۵–۱۰۰: قبولی قوی | ۷۵–۸۴: قبولی | ۶۰–۷۴: قبولی مشروط | کمتر از ۶۰: اصلاح بخش ضعیف

بسته تحویل (همه مشخص و قابل بازبینی)

- فایل پاسخ ۱۰ سؤال ارزیابی پیش از شروع، همراه میزان اطمینان
- فایل conversions.md شامل ۲۰ تبدیل و روند محاسبه
- کد توابع و حداقل ۲۰ نتیجه آزمون
- جدول پیش‌بینی در برابر مشاهده رجیستر فرضی و شواهد Debug
- تاریخچه Git با حداقل یک Commit جدید معنادار برای مأموریت ۰۲
- پاسخ کوتاه پایان هفته و فهرست ابهام‌های باقی‌مانده

پاسخ پایان هفته در ۵ خط

چه چیزی ساختم؟ کجا اشتباه کردم؟ چطور ایراد را محدود کردم؟ با چه مدرکی فهمیدم خروجی درست است؟ چه چیزی را باید در هفته بعد روشن کنم؟

> **قانون پاسخ‌گویی:** عکس یا عبارت «کار کرد» به‌تنهایی کافی نیست. برای هر مدرک یک جمله بنویس که چه ادعایی را اثبات می‌کند. در نقدها، خطای فنی از شخصیت فرد جداست و بازخورد خصوصی ارائه می‌شود.

۸. منابع و اصطلاحات ضروری

قرار نیست همه منابع را کامل بخوانی؛ برای هر سؤال، بخش مربوط را پیدا کن و جواب را با اجرای خودت بررسی کن.

منبع	چه چیزی از آن لازم است؟
مرجع زبان C / پیش‌نویس عمومی C11	تعریف نوع داده صحیح، تبدیل نوع‌ها و عملگرهای بیتی. open-std.org/jtc1/sc22/wg14/www/docs/n1570.pdf
cppreference – C operators	قواعد << >> ~ ^ | & و تفاوت با عملگرهای منطقی. en.cppreference.com/w/c/language/operator_arithmetic
SEI CERT C – INT13-C	دلیل ترجیح operandهای unsigned در عملیات بیتی.
IAR C-SPY	مشاهده متغیر، اجرای مرحله‌ای، Watch و پنجره Memory.
Git Book	فصل‌های مقدماتی Repository، Commit و Log. git-scm.com/book/fa/v2

فرهنگ واژه یک‌خطی

Build: ساخت خروجی از کد منبع. **Rebuild**: بازسازی دوباره خروجی‌ها. **Debugger**: ابزار مشاهده رفتار اجرای برنامه. **Evidence**: شواهدی که ادعای فنی را اثبات می‌کنند. **Gate**: تصمیم عبور بر مبنای مهارت، نه تاریخ. **Reserved bit**: بیتی که نباید بدون مجوز مستندات تغییر کند.

چک‌لیست آخر: آیا می‌توانم یک مقدار ۸ بیتی را بخوانم؟ آیا قبل از اجرا مقدار جدید را درست حدس می‌زنم؟ آیا چهار تابع را خودم توضیح می‌دهم؟ آیا می‌توانم ثابت کنم بیت‌های غیرهدف دست‌نخورده‌اند؟